國立臺灣師範大學資訊工程學系

113 資訊專題研究（二）期末書面報告

實作 Rust 微核心系統：探討作業系統設計的模組化與安全性

A Rust-based Microkernel: Exploring Modularity and Safety in OS Design

指導教授 王超 教授

學生 陳彥誠 撰

中華民國 114 年 6 月

Abstract

This project aims to investigate how the features of the Rust language can improve modularity and security in the development of a safe operating system. By implementing an actual operating system, we explore how Rust's type system and memory safety can ensure isolation and prevent issues like buffer overflows or use-after-free bugs commonly found in C-based systems. The goal is to develop a microkernel operating system that prioritizes reliability, security, and performance.

1 Research Motivation and Background

1.1 Motivation

The initial motivation came from reading a paper [4] exploring the impact of high-level languages on writing operating system kernel. Traditionally, kernels have been written in C due to its high performance and simplicity. However, writing safe code in C requires considerable expertise to avoid memory management errors and undefined behavior, which remain leading causes of kernel vulnerabilities and reliability issues. For example, in 2017, the Common Vulnerabilities and Exposures (CVE) database listed 217 vulnerabilities affecting the Linux kernel, two-thirds of which could be attributed to the use of unsafe languages such as C [3].

Rust is a modern programming language that offers a practical and safe alternative to C. Following recent developments in the Linux kernel and the growing adoption of Rust, we became increasingly aware of Rust's unique approach to type and memory safety. After studying the RedLeaf paper, which demonstrated how Rust's safety features could be applied to microkernel architectures for even greater modularity and isolation, our aim is to explore how Rust's features can impact OS design in practice.

The original plan was to implement a microkernel operating system. However, given the scope and limited time available, we prioritized implementing essential kernel features such as memory management, process & thread handling, and a basic shell. Completing the microkernel architecture remains an important goal for future development.

1.2 Rust

Rust is a programming language designed with safety and performance at its core. Unlike traditional languages such as C, Rust provides built-in guarantees for type and memory safety without relying on garbage collection. Rust's memory safety is achieved through a system of ownership, borrowing, and lifetimes, ensuring each memory location has a single owner or is safely shared, avoiding undefined behavior. By avoiding garbage collection, Rust enables low-level programming without the memory management overhead commonly associated with other safe languages, such as Go.

Another standout feature of Rust is its ability to enforce thread safety. By ensuring that mutable references cannot be shared across threads unless explicitly wrapped in concurrency-safe primitives, Rust prevents data races at compile time. This makes it particularly well-suited for low-level system development, where concurrency and resource management are critical concerns. Rust's emphasis on safety and reliability makes it an ideal choice for developing a secure and modular operating system.

1.3 RISC-V

We chose RISC-V over the x86 instruction set due to its simplicity and openness. RISC-V's clean and minimal design makes it an excellent platform for learning and developing operating systems. Additionally, its open architecture provides flexibility for experimentation, aligning with our project's goal of building a modern, modular microkernel.

2 Literature Review/Related Works

Related works about Rust OS illustrate the growing interest in leveraging Rust's safety and performance features for operating system development. Rust-based operating systems, such as Theseus OS [6] and Redox [11], showcase the potential of Rust type systems, ownership model and memory security guarantees to address the challenges inherent in low-level system programming. Also microkernels such as MINIX 3 [1], seL4 [2], and Genode [12] emphasize the ability of modular architecture to isolate components, minimize kernel responsibilities, and improve fault tolerance.

The RedLeaf operating system [7] is a Rust-based microkernel operating system that uses Rust's type and memory safety feature to realize isolation without relying on hardware address spaces. This demonstrates that Rust is not only a safer alternative to C but also offers more powerful abstractions for building secure and efficient systems.

The xv6 operating system [5] is a simple, Unix-like teaching operating system developed for MIT's 6.1810 course. Written in C and ported to the RISC-V architecture, xv6 serves as a reference implementation for studying fundamental operating system concepts such as processes, memory management, file systems, and device drivers.

Another set of teaching operating systems implemented in Rust, including blog_os [9], rCore [8], and Tacos [10], demonstrate how Rust can be used to build low-level system software while benefiting from memory and type safety. These projects served as valuable references during the development of this work.

3 Development Tools and Software/Hardware Requirements

3.1 QEMU and OpenSBI

To support system-level software development, we utilize tools such as the QEMU emulator to simulate RISC-V hardware. Using QEMU allows us to test and debug the operating system in a controlled environment without requiring physical hardware. Furthermore, we integrate OpenSBI (Open Supervisor Binary Interface) as a foundational firmware layer. OpenSBI significantly reduces the effort required to build a custom bootloader by handling essential low-level tasks, such as hardware initialization and control transfer to the kernel.

3.2 Requirements

The following environment was used to build and test the system:

- **Operating System**: Arch Linux
- **CPU Architecture**: x86_64
- **Rust Toolchain**:
  - Rust: 1.89.0-nightly
  - Cargo: 1.89.0-nightly
  - Additional components:
    * `cargo-binutils`
    * `llvm-tools-preview`
    * `rust-src`
- **Emulator**: QEMU

To cross-compile for the RISC-V architecture and run the kernel, the following tools are required:

- **riscv64 toolchain**: e.g., `riscv64-unknown-elf-gcc, riscv64-unknown-elf-gdb`
- **OpenSBI**: Used as the firmware layer to boot the kernel

4 System Architecture

This section describes the boot process, the structure of the system, and how control flows between user and kernel space. Our system is composed of memory management, process and thread control, and system call handling.

4.1 Boot Process

The system begins execution in machine mode using OpenSBI, which initializes the hardware and passes control to the kernel in supervisor mode. The kernel entry point sets up the stack, initializes the trap vector, and memory. After system initialization, the kernel loads user applications into memory and starts the first user process and shell.

4.2 Kernel Components

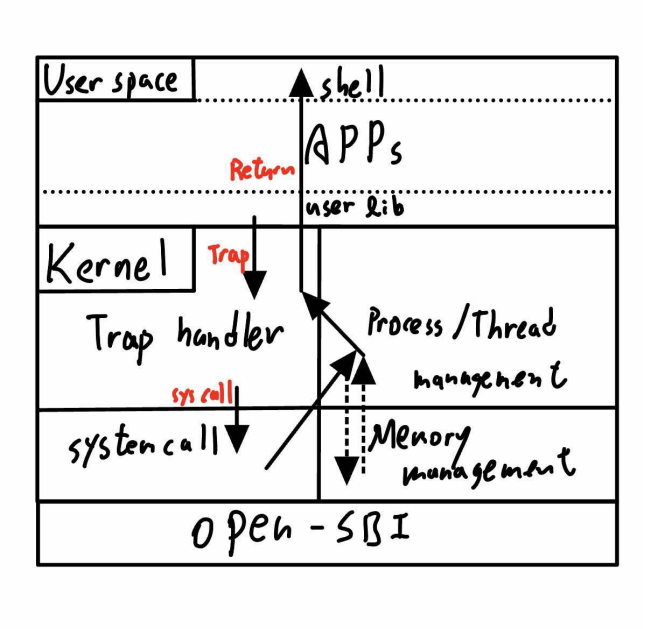

Figure 1: System components and trap flow.

The kernel is divided into several key modules:

- **Memory Manager**: Manages physical and virtual memory through a page table abstraction and allocators, also create address space isolation.
- **Process & Thread Manager**: Handles process & thread creation, scheduling, and process exit/wait.
- **Trap and Exception Handler**: Catches exceptions and system calls from user space, dispatching them to the correct kernel functions.
- **User Shell**: A basic interactive shell for launching user programs.

5 System Functions

5.1 Boot loader

We use OpenSBI to configure the hardware and manage the control transfer to the kernel. Our approach includes setting up the trap vector and stack pointer, defining the memory layout, and organizing the code, data, and stack sections to ensure OpenSBI can load the kernel correctly. Additionally, we load the application into memory and clear the BSS section. We reference Tacos [10] for the usage of opensbi.

5.2 Memory management

We reference rCore [8], xv6 [5] for the design of memory management.

5.2.1 Page table

We use RISC-V's SV39 paging scheme. And use stack-based page frame allocator to manage the page resources.

5.2.2 address space

For each process, we manage their own address space. By initialize different page table for each process, we create the isolation so one process cannot access another's address space.

5.3 Process/Thread management

We reference xv6 [5] for the design of some system call and user shell, and part of context switching(trampoline).

Process Control Block (PCB). pid, parent, children, threads reference, and its address space page table.

Thread Control Block (TCB). Each thread stores the kernel stack, TrapFrame, current state, and a tid. Threads that belong to the same process share the address space from the PCB.

Scheduler. We employ a simple round-robin scheduler. Runnable TCBs are kept in a mutex-protected vector. The scheduler is invoked

- explicitly via the `yield` system call, or
- implicitly on every timer interrupt.

5.3.1 Context switching

We use a trampoline page mapped at the top of each address space to handle traps under paging. And put the trap frame(context) under it for context switching.

5.3.2 Trap handler

- **read, write**: Provide input and out put for user shell.
- **exit**: Process exit, early recycle the resources.
- **yield**: Yield its execution to another process.
- **fork**: Create child process.
- **exec**: Run different app.
- **wait**: Wait for child process exit and free all its resources.

Additionally, the kernel handles two types of exceptions and interrupt:

- **Some memory faults**: Occurs when an application attempts an invalid memory store operation.
- **IllegalInstruction**: Occurs when an application executes an unsupported or privileged instruction in user mode.
- **Interrupt**: Support interrupt for scheduling.

5.4 Demo

5.4.1 Fork

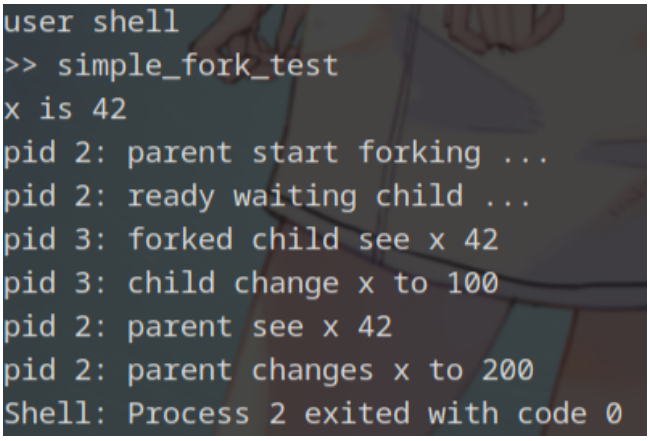

Figure 2: fork demo

We can see that user types a app in shell, and exec the test. The test is that parent fork a child, and both the parent and child can see the origin x, and change their x independently. Indicate that memory space, fork, and shell are successfully implemented.

5.4.2 Exec & Wait

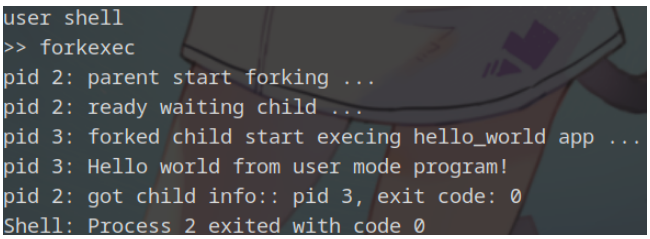

Figure 3: exec demo

We can see the parent fork a child process, then the child process exec to hello_world program. Finally, parent wait for the child. Shows that exec & wait system call are correctly implemented.

5.4.3 Thread create

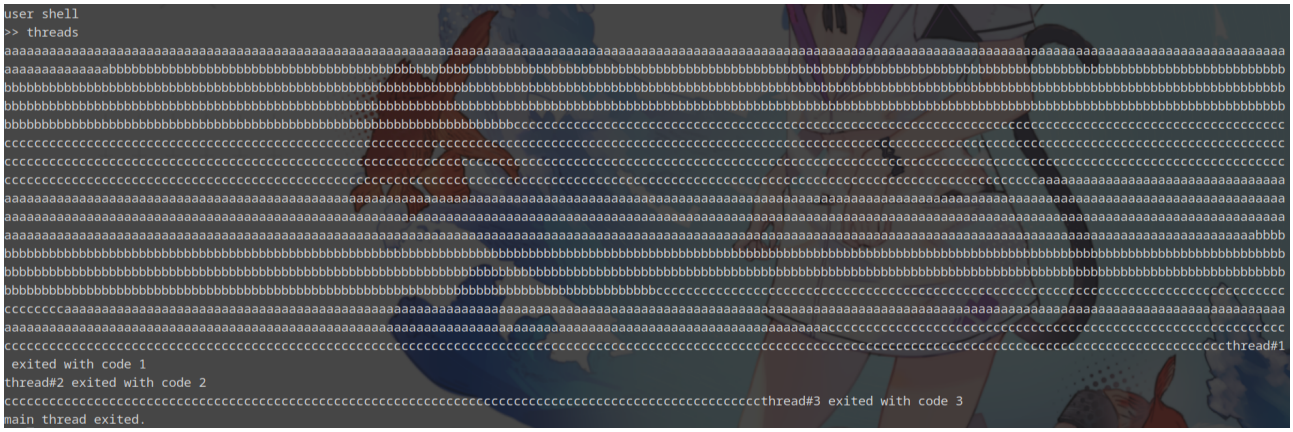

Figure 4: thread create demo

We can see three thread printing a, b, c asynchronously. Shows that our thread are successfully implemented.

6 System Features

We use an example of a stack-based page frame allocator to show how Rust's feature enable safe OS design compared to traditional C-based approaches.

1. Thread safe

   Concurrency safety is one of Rust's core features. Rust disallows unprotected mutable global variables, preventing data races at compile time. With `lazy_static`, we can define global variables that are safely initialized only once, even in concurrent contexts. By wrapping such globals in a `Mutex`, Rust provides a convenient and reliable way to ensure safe access from multiple threads, making concurrency management both safe and ergonomic.

```
lazy_static! {
    pub static ref FRAME_ALLOCATOR: Mutex<FrameAllocatorImpl> =
        Mutex::new(FrameAllocatorImpl::new());
}
```

2. Error handling

   Often, programmers may be tempted to ignore the possibility of an object being null, which can lead to runtime errors. Rust forces explicit handling of such cases through the `Option` type, eliminating null pointer dereferences and making potential edge cases part of the function's type signature.

```
impl FrameAllocator for StackFrameAllocator {
...
    fn alloc(&mut self) -> Option<PhysPageNum> {
        if let Some(ppn) = self.recycled.pop() {
            Some(ppn.into())
        } else if self.current == self.end {
            None
        } else {
            self.current += 1;
            Some((self.current - 1).into())
        }
    }
...
}
```

3. Type system and memory safety

   Rust is renowned for its strong type system and unique approach to memory safety. By wrapping a `PhysPageNum` in the `FrameTracker` type, we can bind the memory lifetime of a page frame directly to the lifetime of the variable, eliminating the need for manual `malloc` and `free`, as well as avoiding garbage collection. This design prevents memory bugs such as leaks and double frees without introducing additional overhead. Furthermore, by implementing the `Debug` and `Drop` traits for `FrameTracker`, we enable convenient debugging output and ensure that page frames are automatically released when no longer needed.

```
pub struct FrameTracker {
    pub ppn: PhysPageNum,
}

impl FrameTracker {
    pub fn new(ppn: PhysPageNum) -> Self {
        // page cleaning
        let bytes_array = ppn.get_bytes_array();
        for i in bytes_array {
            *i = 0;
        }
        Self { ppn }
    }
}

impl Debug for FrameTracker {
    fn fmt(&self, f: &mut Formatter<'_>) -> fmt::Result {
        f.write_fmt(format_args!("FrameTracker:PPN={:#x}", self.ppn.0))
```

```
    }
}

impl Drop for FrameTracker {
    fn drop(&mut self) {
        frame_dealloc(self.ppn);
    }
}
```

7 Future Research Directions

We have successfully implemented the basic functionalities of the operating system, including the boot loader, address space isolation, user-space shell, and process and thread management. As the next step, we plan to implement an inter-process communication (IPC) mechanism and a more sophisticated scheduler. These components will serve as the foundation for designing a true microkernel architecture. Ultimately, our goal is to build a fully functional microkernel operating system, integrating additional components such as a file system to create a more complete and usable system.

References

[1] Jorrit N. Herder et al. "MINIX 3: a highly reliable, self-repairing operating system". In: *SIGOPS Oper. Syst. Rev.* 40.3 (July 2006), pp. 80–89. ISSN: 0163-5980. DOI: 10.1145/1151374.1151391. URL: https://doi.org/10.1145/1151374.1151391.

[2] Gerwin Klein et al. "seL4: formal verification of an OS kernel". In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*. SOSP '09. Big Sky, Montana, USA: Association for Computing Machinery, 2009, pp. 207–220. ISBN: 9781605587523. DOI: 10.1145/1629575.1629596. URL: https://doi.org/10.1145/1629575.1629596.

[3] Abhiram Balasubramanian et al. "System Programming in Rust: Beyond Safety". In: *SIGOPS Oper. Syst. Rev.* 51.1 (Sept. 2017), pp. 94–99. ISSN: 0163-5980. DOI: 10.1145/3139645.3139660. URL: https://doi.org/10.1145/3139645.3139660.

[4] Cody Cutler, M. Frans Kaashoek, and Robert T. Morris. "The benefits and costs of writing a POSIX kernel in a high-level language". In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 89–105. ISBN: 978-1-939133-08-3. URL: https://www.usenix.org/conference/osdi18/presentation/cutler.

[5] Robert Morris, Russ Cox, and Frans Kaashoek. *Xv6, a simple Unix-like teaching operating system*. https://pdos.csail.mit.edu/6.828/2019/xv6.html. 2019.

[6] Kevin Boos et al. "Theseus: an Experiment in Operating System Structure and State Management". In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 1–19. ISBN: 978-1-939133-19-9. URL: https://www.usenix.org/conference/osdi20/presentation/boos.

[7] Vikram Narayanan et al. "RedLeaf: Isolation and Communication in a Safe Operating System". In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 21–39. ISBN: 978-1-939133-19-9. URL: https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram.

[8] rCore Developers. *Rust version of THU uCore OS. Linux compatible.* https://rcore-os.cn/rCore-Tutorial-Book-v3/. 2022.

[9] Philipp Oppermann. *Writing an OS in Rust*. https://os.phil-opp.com/. 2022.

[10] Tacos Developers. *Write your own operating system with Rust!* https://pku-tacos.pages.dev/introduction/. 2024.

[11] Redox Project Developers. *Redox - Your Next(Gen) OS*. URL: http://www.redox-os.org.

[12] Norman Feske. *Genode operating system frame- work*. URL: https://genode.org/documentation/genode-foundations-19-05.pdf.